

รายงาน Project 1: Socket Programming

เรื่อง โพรโทคอลในเกมออนไลน์

นายเอื้ออังกูร แสนโคตร รหัสนิต 6610402299

ระบบจำลองเกม 2D แบบเรียลไทม์ที่รองรับผู้เล่นใช้งานพร้อมกัน 4 คนต่อ 1 ห้อง มุ่งเน้นการตรวจสอบเสถียรภาพในการสื่อสารข้อมูล พิกัดผ่านทั้งเครือข่ายอินเทอร์เน็ตสาธารณะ (WAN) เครือข่ายเฉพาะที่ (LAN) เป็นระบบจำลองเพื่อพิสูจน์การทำงานของสถาปัตยกรรมเครือข่าย มุ่งเน้นเฉพาะการซิงโครไนซ์ข้อมูลพิกัดการเคลื่อนที่ของตัวละคร 2D โดยคัดลอกองค์ประกอบที่ไม่เกี่ยวข้องกับการทดสอบโครงข่าย เช่น ระบบเกมเพลย์เชิงลึก แอนิเมชันขั้นสูง ทรัพยากรฉาก (Assets) และระบบเสียงประกอบ (SFX/BGM) เพื่อคงขอบเขตการทำงานไว้ที่ระดับ โครงสร้างพื้นฐานของระบบเครือข่าย สภาพแวดล้อมทดสอบสำหรับการเชื่อมต่อและโต้ตอบระหว่างผู้เล่น 4 คนพร้อมกันผู้เล่นสามารถควบคุม การเคลื่อนที่ของตัวละครเรียลไทม์ทั้งในกรณีที่ผู้เล่นอยู่ต่างเครือข่ายกัน (ออนไลน์) และกรณีที่อยู่ภายใต้เครือข่ายวงเดียวกัน (โลคอล)

วัตถุประสงค์

- เพื่อทดสอบและยืนยันการทำงานของโปรโตคอลรับส่งข้อมูลพิกัดตำแหน่งตัวละครแบบเรียลไทม์
- เพื่อประเมินประสิทธิภาพของสถาปัตยกรรม Client-Server บนเซิร์ฟเวอร์ VPS สำหรับโหมดออนไลน์
- เพื่อประเมินประสิทธิภาพของสถาปัตยกรรม Peer-to-Peer / Listen Server บนโครงข่าย LAN สำหรับโหมดโลคอล

คุณลักษณะ

- รองรับผู้เล่นสูงสุด 4 คนต่อ 1 ห้อง
- ส่งข้อมูลพิกัดตัวละคร 2D ของผู้เล่นทุกคนอย่างต่อเนื่องและตรงกันแบบเรียลไทม์
- สถาปัตยกรรมโหมดออนไลน์ Online Mode - Client-Server:
 - ใช้ VPS เป็นเซิร์ฟเวอร์กลาง
 - โคลเอนต์ทั้งหมดส่งข้อมูลพิกัดไปยัง VPS เพื่อประมวลผลและกระจายข้อมูลไปยังผู้เล่นอื่นในห้อง
 - ทำงานผ่านพอร์ตเฉพาะที่กำหนดไว้ (พอร์ต 42512)
- สถาปัตยกรรมโหมดโลคอล (Local Mode - Peer-to-Peer / Listen Server):
 - ผู้สร้างห้องทำหน้าที่เป็นทั้งโฮสต์ (Host) และโคลเอนต์ในตัว
 - ใช้ระบบค้นหาเครือข่ายผ่านการ Broadcast ภายในวง LAN เดียวกัน
 - ผู้เล่นอื่น (Peers) สามารถค้นหาและเชื่อมต่อตรงเข้าสู่โฮสต์ได้ทันทีโดยไม่ต้องพึ่งพาเซิร์ฟเวอร์ภายนอก

ออกแบบ Application-Layer Protocol

ตั้งชื่อ Protocol ว่า GUDP ใช้โปรโตคอล UDP (User Datagram Protocol) พอร์ต 42512 เพราะต้องการความเร็วในการรับส่งข้อมูล

ตารางแสดงโครงสร้าง Header (ขนาด 15 Bytes)

Field	Size (Bytes)	Description
Magic Number	2	ตัวตรวจสอบความถูกต้องของแพ็กเก็ต (0x8750)
Session ID	4	รหัสการเชื่อมต่อ ป้องกันข้อมูลแปลกปลอม
Player ID	2	หมายเลขประจำตัวผู้เล่น
Packet Type	1	ประเภทคำสั่ง
Target Type	1	เป้าหมาย (0x00=SERVER, 0x01=ROOM)
Sequence Num	4	ลำดับแพ็กเก็ตกันการส่งซ้ำสลับกัน
Payload Length	1	ขนาดข้อมูล Payload ที่จะตามมา (0-255)

- H (unsigned short) = **2 Bytes** (ใช้กับ MAGIC_NUMBER)
- I (unsigned int) = **4 Bytes** (ใช้กับ SESSION_ID)
- H (unsigned short) = **2 Bytes** (ใช้กับ PLAYER_ID)
- B (unsigned char) = **1 Byte** (ใช้กับ PACKET_TYPE)
- B (unsigned char) = **1 Byte** (ใช้กับ TARGET_TYPE)
- I (unsigned int) = **4 Bytes** (ใช้กับ SEQUENCE_NUM)
- B (unsigned char) = **1 Byte** (ใช้กับ PAYLOAD_LENGTH)

ตารางแสดง Packet Type

Hex Type	Command	Message / Action	Response
0x01	CREATE_ROOM	ขอสร้างห้องใหม่ พร้อมส่งชื่อและสี	ตอบกลับ Room Code และ State Update
0x02	JOIN_ROOM	ขอเข้าร่วมห้อง พร้อมรหัสห้อง ชื่อ สี	ตอบกลับ Room Code หรือ Error
0x08	MOVE	ส่งพิกัด X, Y ปัจจุบัน	กระจาย (Broadcast) ให้คนอื่นในห้อง
0x09	STATE_UPDATE	สถานะล่าสุดของทุกคนในห้อง	แจ้งอัปเดตสถานะให้ Client
0x0A	ERROR	แจ้งข้อผิดพลาด	Client เปลี่ยน State เป็นหน้าต่าง Error
0x0B	LEAVE_ROOM	แจ้งการออกจากห้อง	Server ลบผู้เล่นและส่ง STATE_UPDATE ใหม่
0x0C	LIST_ROOMS	ร้องขอข้อมูลห้องที่กำลังเปิดทั้งหมด	รายชื่อห้อง, จำนวนคน, และสีที่ใช้แล้ว

ตารางแสดง Status code

Status Code	Status Phrase	Description
0x00	OK	ดำเนินการสำเร็จ
0x01	ROOM_NOT_FOUND	ไม่พบห้องที่ค้นหา
0x02	ROOM_FULL	ห้องเต็ม (ครบ 4 คนแล้ว)
0x03	ERROR	ข้อผิดพลาดทั่วไป
0x04	NAME_TAKEN	ชื่อถูกใช้งานแล้วในห้องนี้
0x05	COLOR_TAKEN	สีตัวละครถูกใช้งานแล้วในห้องนี้
0x06	MAX_ROOMS_REACHED	เซิร์ฟเวอร์มีห้องเต็มขีดจำกัดแล้ว (3 ห้อง)

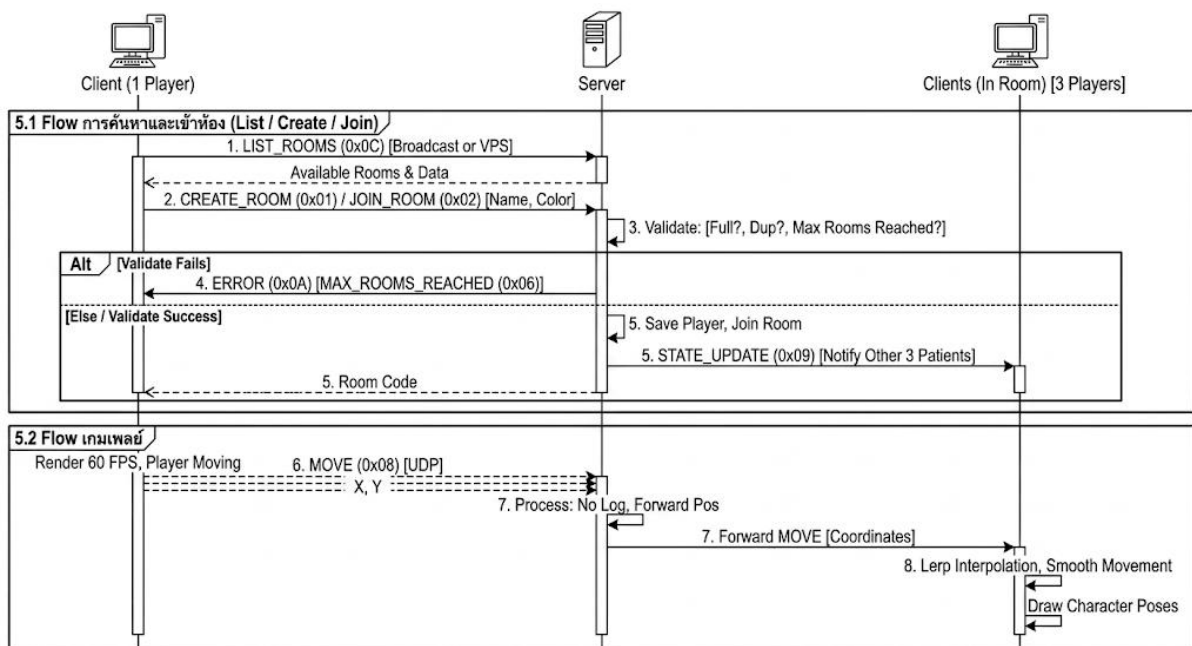
การจัดการข้อมูลของโปรแกรม โดยแบ่งออกเป็น 2 แบบ ดังนี้

1. ข้อมูลกึ่งถาวร ข้อมูลกึ่งถาวรจะถูกจัดเก็บบนหน่วยความจำ (RAM) ทั้งฝั่ง Server และ Client กระทำผ่านการสร้างโครงสร้างข้อมูลประเภท Dictionary ร่วมกับ Class ทัวไปฝั่งเซิร์ฟเวอร์ และใช้งานร่วมกับ @dataclass (PlayerInfo) ในฝั่งไคลเอนต์ ข้อมูลชุดนี้จะถูกจองพื้นที่ใน RAM ทันทีที่ตัวแปรถูกสร้างและจะถูกล้างทิ้งเมื่อผู้เล่นออกจากห้องหรือโปรแกรมปิดตัวลง โดยไม่มีการเขียนข้อมูลใดๆ ลงฮาร์ดดิสก์

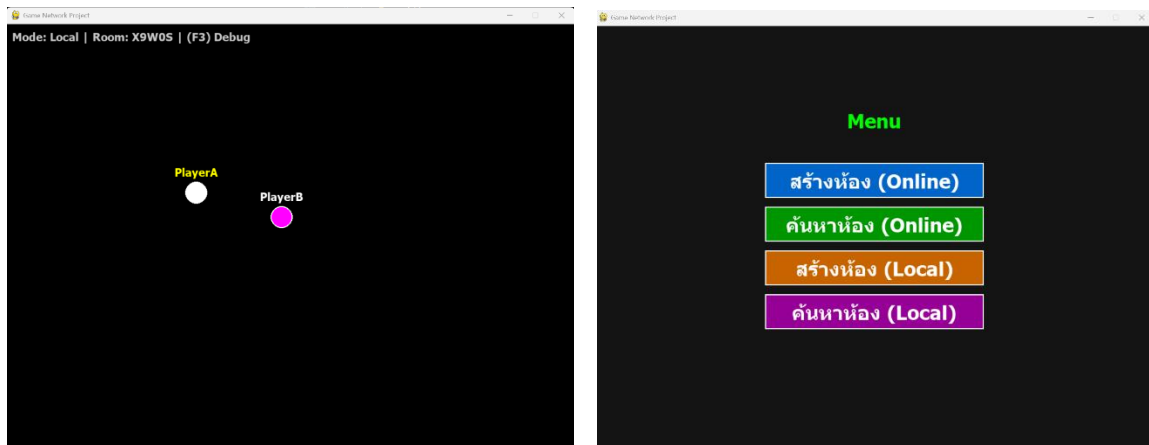
- ตัวแปร: รหัสห้อง (Room Code), รหัสผู้เล่น (Player ID), ชื่อผู้เล่น (Username) และ สีตัวละคร (Color)
- การจัดการ: โหลดเข้าสู่หน่วยความจำและคงสถานะไว้ครบเท่าที่ผู้เล่นยังอยู่ในห้อง ระบบ UI จะอ้างอิงตัวแปรชุดนี้เพื่อวาดรายชื่อและสีของตัวละครที่ถูกต้องลงบนหน้าจอ
- การอัปเดต: เกิดขึ้นเฉพาะเมื่อมี Event สำคัญเท่านั้น เช่น ผู้เล่นสร้างห้องใหม่ เข้าร่วมห้อง หรือออกจากห้อง โดยระบบจะอัปเดตข้อมูลให้ตรงกันผ่านแพ็กเก็ตคำสั่ง STATE_UPDATE

2. ข้อมูลที่มีความเปลี่ยนแปลงสูง ข้อมูลประเภทนี้มีการอัปเดตตลอดเวลาในทุกๆ เฟรมเมื่อ Client ได้รับแพ็กเก็ตคำสั่ง MOVE จากเซิร์ฟเวอร์ โปรแกรมจะทำการ "เขียนทับ" ค่าตำแหน่งเป้าหมายเดิม (target_x, target_y) ในหน่วยความจำทันที เพื่อให้ระบบนำค่าใหม่ไปใช้คำนวณการเคลื่อนที่ในเฟรมถัดไป ข้อมูลตำแหน่งที่ถูกใช้แล้วจะถูกทำลายทิ้งโดยระบบจัดการหน่วยความจำของภาษา Python

- ตัวแปร: พิกัดตำแหน่งการเดิน (X, Y)
- การจัดการ: เป็นการประมวลผลแบบใช้แล้วทิ้งไม่มีการบันทึกประวัติการเดินย้อนหลังและไม่จัดเก็บเป็นโครงสร้างข้อมูลถาวร เพื่อให้ระบบกินทรัพยากรน้อยที่สุดและตอบสนองได้แบบเรียลไทม์



แผนภาพลำดับการทำงาน



ตัวอย่าง UI ของโปรแกรม

ตัวอย่างการส่งแพ็กเก็ต MOVE (HEX จำนวน 23 Bytes)

87 50 00 00 00 01 00 02 08 00 00 00 00 0F 08 43 C8 00 00 43 96 00 00
--

- 87 50 (2 Bytes): Magic Number ของโปรโตคอลเพื่อยืนยันเกม (0x8750)
- 00 00 00 01 (4 Bytes): Session ID ของผู้เล่น (หมายเลข 1)
- 00 02 (2 Bytes): Player ID (ผู้เล่นหมายเลข 2)
- 08 (1 Byte): Packet Type (0x08 = คำสั่ง MOVE)
- 00 (1 Byte): Target Type (0x00 = SERVER)
- 00 00 00 0F (4 Bytes): Sequence Number (แพ็กเก็ตลำดับที่ 15 / 0x0F)
- 08 (1 Byte): Payload Length (มีข้อมูลตามมาอีก 8 Bytes)
- 43 C8 00 00 (4 Bytes): ข้อมูล Payload พิกัดแกน X (Float = 400.0)
- 43 96 00 00 (4 Bytes): ข้อมูล Payload พิกัดแกน Y (Float = 300.0)

ฝั่งรับ Receive :

1. อ่านเมื่อ Client/Server ได้รับข้อมูลจาก Socket ผ่านฟังก์ชัน recvfrom() จะทำการดึงข้อมูล 15 Bytes แรกออกมาก่อน (Header)
2. ตรวจสอบความถูกต้อง นำ 2 Bytes แรกมาเช็คผ่าน struct.unpack หากไม่ใช่ค่า 87 50 โปรแกรมจะทลายทิ้ง เพื่อกันแพ็กเก็ตขยะ
3. แยกข้อมูลดึงค่า Player ID และประเภทคำสั่ง เพื่อระบุว่าใครเป็นคนส่งและต้องการทำอะไร (ในกรณีนี้คือคำสั่ง MOVE 0x08)
4. อ่าน Payload ดูค่า Length (0x08) แล้วดึงข้อมูล 8 Bytes ถัดไปเพื่อแปลงกลับเป็นพิกัดแกน X และ Y ด้วย struct แบบ Float
5. ส่งข้อมูล Packet และ IP Address เข้าฟังก์ชัน _handle_network_response() นำค่า X, Y ไปอัปเดตตำแหน่งเป้าหมาย (client_state.updatePosition) ในหน้าจอเกมเพื่อให้ตัวละครเคลื่อนที่